

AAPO VANHAINEN

<https://aapovanhainen.github.io/portfolio/>
aapo.vanhainen@gmail.com
[linkedin.com/in/aapo-vanhainen](https://www.linkedin.com/in/aapo-vanhainen)

Game developer with a focus on game audio and interactive sound design, experienced in Unity projects including commercial releases, professional training, and student productions. Hands-on experience with sound effect creation, FMOD integration, spatial and event-based audio, and gameplay-driven audio implementation. Additional strengths in gameplay programming, QA, report-writing, Office tools and technical problem-solving, with an academic foundation. More information, tools, design documents and other skills can be found in the portfolio.

WORK EXPERIENCE

Exsanguination

Audio Design • Audio Production • Foley Sound Production • Audio Editing

- Created and edited sound effects for combat, movement, and in-game actions
- Recorded and produced Foley sounds for character and environmental interactions
- Implemented audio systems and sound logic in Unity
- Produced and maintained game and audio documentation (GDD, ADD)
- Programmed player movement and combo-based combat systems; designed two enemy AI types and mentored a junior programmer

Versebound

Audio Production • Audio Editing • Audio programming

- Produced, edited and implemented sound effects
- Designed and implemented a dynamic environmental audio system
- Contributed to gameplay programming, feature implementation, QA, game balance and marketing

Neverance

Audio Design • Audio Programming

- Built an FMOD Audio Manager to control global SFX and music volumes via VCAs
- Ink-based dialogue audio system with character-by-character typing sounds
- Implemented UI and gameplay sound effects with reusable, scalable audio logic
- Documented audio and dialogue systems for internal use and future development

SKILLS

Game Audio

- Audio Editing & Iteration
- Audio-to-gameplay feedback design
- FMOD (events, parameters, Unity integration)
- Sound Effects & Foley Design

Tools

- Agile tools: Jira, Miro
- Audacity
- FMOD
- Office tools
- Reaper
- Splice
- Use of assets (free/paid)

EDUCATION

Bachelor of Business Information Technology — Game Production

Jyväskylä University of Applied Sciences

GPA: 4.5 / 5
12/2025

- Researched Foley production practices in indie game development
- Developed a practical analysis tool for audio production feasibility
- Thesis:
<https://www.theseus.fi/handle/10024/908930>
- Foley sound analyzing tool:
<https://github.com/aapovanhainen/Foley-Sound-Analyzing-Tool>

PROJECTS

Exsanguination • 09/2024 - 12/2024

Never Late Production

Audio was one of my primary focus areas in the Exsanguination project, alongside game design. My responsibilities covered **sound effect design, Foley recording, audio editing, music implementation, and in-engine audio systems**, with the goal of supporting fast-paced melee combat and reinforcing gameplay feedback.

Sound effects as sound files and as a video available at the end, as well as Audio Design Document.

Sound Effects & Foley Production

The majority of the game's sound effects were **custom-recorded Foley**, created specifically for Exsanguination. Recording sessions were conducted in a studio environment using practical props such as training swords, metal tools, footwear, and improvised materials to achieve grounded and reactive sounds for combat and movement.

- Sword equip and swing sounds with multiple swing variations
- Player movement sounds such as jumping, dashing, sliding, and parrying
- Enemy alert sounds, including bat screeches and combat reactions
- Grappling hook interactions and impact sounds
- Boss vocalizations and chants for special attacks

All recorded material was edited in **Audacity**, where I handled noise reduction, normalization, layering, reverb, and variation creation. Combat sounds in particular were produced with **multiple variants** to avoid repetition during rapid combo attacks. In addition to recorded Foley, selected sounds were sourced from online libraries and edited to match the game's sonic style and naming conventions.

Audio Documentation & Asset Organization

To support production and collaboration, I created and maintained:

- An **Audio Design Document (ADD)** defining audio direction, usage rules, and naming conventions
- A **sound effects Excel sheet** tracking required sounds, priorities, recording status, and implementation readiness

These documents helped ensure consistent asset management and smooth integration between audio, programming, and design throughout development.

Music Implementation & Mixing

I worked closely with external music producers to integrate level and boss music into the game. My responsibilities focused on **technical implementation rather than composition**, including:

- Implementing level and boss music in Unity

- Creating a custom **MusicManager system** to control intros, looping, and transitions
- Setting up **Unity Audio Mixers** and volume groups
- Implementing player-adjustable volume sliders for music and sound effects
- Balancing in-game audio levels to maintain clarity during combat

Boss music was implemented with multiple intensity phases, while level music required manual editing and looping due to production constraints. All music behavior was handled through custom scripting and mixer routing to ensure consistent playback across gameplay states

In-Engine Audio Implementation

Beyond asset creation, I was responsible for **integrating all audio systems into gameplay**, ensuring that sound effects and music reacted correctly to player actions, enemy behavior, and game states. This included:

- Triggering sound effects from combat, movement, and enemy logic
- Synchronizing audio with animations and gameplay events
- Testing and refining audio timing and volume balance during playtests
- Iterating based on QA feedback and player testing sessions

The audio work was tightly coupled with gameplay systems, particularly combat and enemy AI, to reinforce responsiveness and player feedback in a fast-paced first-person melee game

The Sound Effects

This is a collection of sound effects I created, where all but Finisher was recorded as Foley sound in a studio: <https://youtu.be/Fpvl1oyYrO0>

List of Sound Effects I created for the game:

Starter Sword / Combat

Starter Sword Equip

Starter Sword Swing 1 variant 1

Starter Sword Swing 1 variant 2

Starter Sword Swing 1 variant 3

Starter Sword Swing 2 variant 1

Starter Sword Swing 2 variant 2

Starter Sword Swing 2 variant 3

Starter Sword Swing 3 variant 1

Starter Sword Swing 3 variant 2

Starter Sword Swing 3 variant 3

Player Combat Parry

Stake / Finisher

Finisher Talk

Stake Throw

Player Movement

Player Slide

Player Landing variant 1
Player Jump
Player Dash
Player Heal variant 1
Player Take Damage

Grappling Hook

Grappling Hook Hit
Grappling Hook Ready variant 2
Grappling Hook Shoot variant 1

UI

Menu Click variant 1
Menu Click variant 2

Enemies / Combat Hits

Enemy Ghoul Alert
Enemy Alert
Enemy Take Damage variant 1
Enemy Take Damage variant 2
Enemy Take Damage variant 3
Enemy Take Damage variant 4
Enemy Take Damage variant 5
Enemy Take Damage variant 6
Enemy Take Damage variant 7

Footsteps – Stone / Concrete

Concrete Footstep variant 1
Concrete Footstep variant 2
Concrete Footstep variant 3

Versebound • 01/2025 - 04/2025

Conifer Digital

Audio played a central role in the Versebound project, supporting its myth-inspired atmosphere, exploration-focused gameplay, and narrative-driven hub environments. My responsibilities covered sound effect production, audio editing, dynamic environmental audio systems, and in-engine implementation, with a strong emphasis on immersion, readability, and systemic audio behavior.

Sound Effects & Audio Production

I created and edited a wide range of sound effects for Versebound, covering gameplay interactions, environmental events, UI feedback, and character-related audio. Sound effects were produced through a combination of original recordings, library-based material, and extensive editing to ensure consistency across the game's soundscape.

Key sound effect categories included:

- Environmental interaction sounds such as bridge collapses, altar activations, and hub-based events
- Character-related sounds including damage reactions
- Ambient creature and environmental sounds (e.g. crows, mosquitoes, wind elements)
- UI and progression feedback sounds such as quest unlocks, bestiary unlocks, and area transitions

All audio assets were edited in Reaper, where I handled trimming, layering, EQ, normalization, spatial balance, and variation creation. Ambient sounds were designed to loop seamlessly and blend naturally with surrounding audio layers, while one-shot sounds were tuned for clarity and gameplay feedback.

Dynamic Environmental Audio Systems

A major focus of my work on Versebound was the creation of a dynamic audio system for the game's hub and environmental areas. Instead of static looping audio, ambient sounds were implemented as multiple layered sources reacting to player position and movement.

This system included:

- Distance-based volume scaling for ambient sounds (e.g. swamp insects, wind elements)
- Multiple looping sound sources placed throughout environments
- Timed delays and staggered playback to avoid repetition and pattern recognition
- Area-specific sound identities to distinguish locations through audio alone

The hub area featured a layered ambient setup that combined environmental loops with ghostly vocal elements inspired by Kalevala poetry, designed to reinforce the game's thematic roots and narrative tone.

In-Engine Audio Implementation

All sound effects and ambient systems were implemented directly in Unity. I was responsible for ensuring that audio behavior reacted correctly to gameplay systems, player movement, and environmental triggers.

This included:

- Integrating one-shot and looping sounds through prefabs and scene placement
- Implementing proximity-based and distance-based audio behavior
- Triggering sound effects through gameplay logic and interaction systems
- Balancing audio levels across environments to maintain clarity without overwhelming the player

Audio implementation was iterated throughout development based on internal testing and feedback, with adjustments made to volume curves, playback timing, and sound layering to improve immersion and consistency.

Audio Documentation & Asset Management

To support structured development and collaboration, I produced and maintained clear audio documentation throughout the project. This included:

- Audio documentation outlining sound categories, usage rules, and implementation logic
- Organized naming conventions and folder structures for all audio assets
- Tracking of implemented and pending sounds to support iteration and QA

This documentation helped maintain consistency across the project and ensured smooth communication between audio, design, and programming responsibilities.

The Sound Effects

Video of a collection of sound effects I created for Versebound:
<https://youtu.be/ZPsqmJUq86k>

List of all sound effects I created for the game (sounds marked with * can be found from the video):

Environment / Level Sounds

- * Bridge Collapse (Level 2 Environment)
- * Ultimate Altar (Unlocking ultimate weapon)

Progression / UI

- * Bestiary Unlock
- * Quest Unlock

Artifacts

- * Markku Artifact
- * Defy Death Artifact

Miniboss

- * Kouvo Miniboss Spawn

Characters

Aino Take Damage
Joukahainen Take Damage
Mosquito Catch (acquire as weapon from level)

Weapons

Northern Breeze Shoot
Northern Breeze Blast

Hub / Ambient Sounds

- * Beer (Hub Ambient)
- Crow (Hub Ambient)
- Hub Ghost (Hub Ambient)
- Swamp (Hub Ambient)
- * Northern Breeze (Hub Ambient)
- * Ilmarinen hammer hit (Hub Ambient)
- * Campfire ambience (Hub Ambient)
- * Beach ambience (Hub Ambient)

Neverance • 10/2025 - 12/2025

Gangster Pukeko Games

Audio work in Neverance focused on technical audio systems, FMOD integration, and interactive sound logic, rather than standalone sound effect creation. My role was to establish scalable audio pipelines and systems that could support a narrative-driven, turn-based game with minimal existing audio direction.

Audio Systems Design & FMOD Integration

I designed and implemented multiple FMOD-based audio systems and integrated them into Unity, despite having no prior FMOD experience at the start of the project. The work emphasized modularity, documentation, and ease of use for the rest of the team.

Key responsibilities included:

- Creating and structuring a full FMOD project with proper event hierarchy, mixers, VCAs, and sound banks
- Integrating FMOD into Unity and configuring live update, bank loading, and project paths
- Developing a centralized FMODAudioManager system to manage sound playback, parameters, and volume control
- Implementing global SFX and music volume control through FMOD VCAs and in-game sliders

Interactive Dialogue Typing Audio System

One of the primary audio features I developed was a dynamic dialogue typing sound system, inspired by narrative-heavy games such as OMORI.

This system included:

- Designing a per-character typing sound effect system using FMOD multi-instruments with randomized pitch and volume
- Integrating FMOD with the Ink dialogue system and TextMeshPro
- Writing custom Unity scripts to trigger audio on each character reveal while preventing audio spam
- Supporting adjustable typing speeds through Ink tags and synchronized audio playback
- The system ensured that dialogue audio reacted directly to narrative pacing and UI behavior, improving readability and player feedback during story sequences.

Gameplay Audio Logic & “Sweetness” System

To support Neverance’s exploration mechanics, I designed and implemented a progressive audio feedback system, referred to as the Sweetness system.

This system:

- Used a global FMOD parameter to control escalating audio feedback during successful exploration sequences
- Dynamically adapted to different pattern lengths and item rarities

- Provided increasing musical feedback as the player successfully chained actions, reinforcing reward and momentum
- Reset correctly on failure states to maintain clarity and consistency
- All logic was handled through Unity scripting combined with FMOD parameters, allowing designers to adjust behavior without modifying code.

Documentation & Knowledge Transfer

A major part of my responsibilities was ensuring that audio systems were understandable and reusable by the rest of the team.

I produced detailed documentation covering:

- FMOD project structure and naming conventions
- Step-by-step guides for creating and integrating new sound events
- Unity-FMOD wiring, parameter usage, and Audio Manager logic
- Ink dialogue system setup and reproduction instructions

These documents enabled the team to extend and maintain audio systems independently after my practical training period.